Tema 1. Bosques aleatorios

1. Descomposición en sesgo/varianza

Existen tres tipos de error:

- **Error de test o generalización**. Generado sobre un conjunto de datos aleatorios diferentes a los usados para el entrenamiento del modelo.

  $$\mathrm{Err}_T = \mathrm{E}\left[L(Y, \hat{f}(X)|T\right]$$

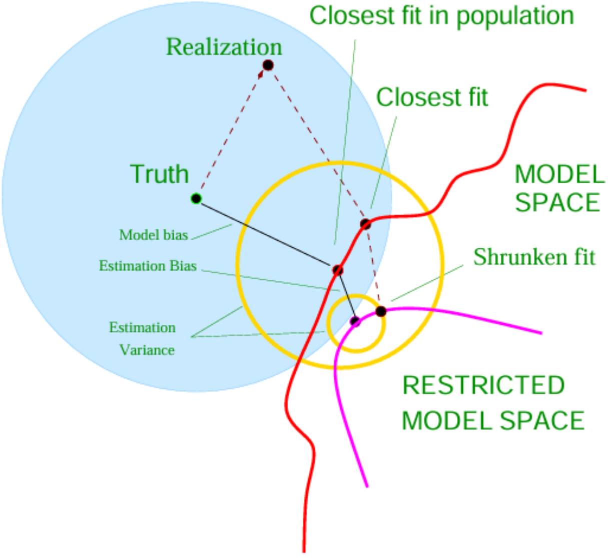
  Donde L es una función de error definida (como el MSE), X e Y es un conjunto junto con su valor asociado.
- **Error de entrenamiento**. Error generado por el modelo sobre el conjunto de entrenamiento. A más ajustado esté el modelo a los datos (sobre ajustado) menor tenderá a ser el error.
- **Error de predicción esperado**. Promedio de todos los posibles errores de test.

  $$\mathrm{Err} = \mathrm{E}[\mathrm{Err}_T]$$

  Este error se puede definir, dado una instancia x_0, en base a la varianza y el sesgo de la proyección y la varianza del ruido, que no se puede evitar de ninguna forma.

  $$\mathrm{Err}(x_0) = \mathrm{Var}(\epsilon) + \mathrm{Bias}^2\left(\hat{f}(x_0)\right) + \mathrm{Var}\left(\hat{f}(x_0)\right)$$

  $Var(\epsilon)$ es la varianza de la función real f. Generalmente, a mayor varianza menor sesgo. O lo que es lo mismo, a mayor flexibilidad (menor sesgo), mayor varianza (**compromiso**).
  - La varianza representa la cantidad en la que variará el error cambiando de dataset.
    - Con varianza alta, un cambio pequeño en los datos puede generar grandes cambios en $\hat{f}$.
  - El sesgo es el error que introduce la función usada para modelar la realizada.
    - Una función lineal es complicada que modele la realidad correctamente, por lo que introducirá mucho seso.
    - A más flexible el modelo, menor sesgo.



El punto “truth” es el punto real. La zona azul es el posible ϵ. El valor “closest fit” es el punto calculado por el mejor modelo posible, que se diferencia del “closest fit in population” por la varianza del modelo.

El cálculo de los errores tiene como objetivo:

- Seleccionar el mejor modelo.
- Evaluar el modelo seleccionado.

Los datos se suelen dividir en tres:

- **Datos de entrenamiento (70%)**. Usados para para entrenar el modelo.
- **Datos de validación (20%)**. Puede ser un subconjunto de los datos de entrenamiento. Se usan para seleccionar el mejor modelo.
- **Datos de test (10%)**. Se usa para evaluar el modelo. Deben tratarse como datos totalmente independientes a los otros dos conjuntos, incluso en términos de escalado de datos.

2. Bootstrap

Se trata de una técnica de re-muestreo en base a una muestra ya obtenida. Dada una muestra de n instancias, se generan B muestras a partir de la original con el mismo número de instancias, por lo que existirán instancias de la muestra original que no existan en la generada.

Esta técnica no asume ninguna distribución sobre los datos. Permite estimar la distribución de los datos cuando el número $B \to \infty$.

3. Bagging

Procedimiento de propósito general para reducir la varianza de un método de aprendizaje estadístico.

Bagging: dado un conjunto de observaciones independientes, cada una con varianza σ^2, la varianza de la media de las observaciones es σ^2/n. Usando la técnica del bootstraping, se pueden generar B conjuntos de entrenamiento, cada uno con un modelo asociado, $\hat{f}^*$, de forma que la predicción para una instancia x se genere tal que

$$\hat{f}_{avg}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b^*(x)$$

Para casos de clasificación, en lugar de hacer la media, se escoge la moda.

Especialmente útil para árboles de decisión y modelos especialmente no lineales. El valor B no produce sobreajuste sea cual sea el valor, aunque se suene usar en torno a 100. El uso de esta técnica reduce la capacidad de interpretación de los árboles al existir una gran cantidad, aunque se puede conocer la importancia de las características usando el RSS (suma residual de cuadrados) en regresión o el índice de Gini en clasificación.

La idea principal de esta técnica es obtener la media de muchos modelos ruidosos pero que no estén sesgados, de forma que se pueda reducir la varianza.

4. Random forest

Consiste en muchos árboles no correlacionados entre sí que generan una única salida a partir de la media de las salidas generadas por cada árbol de decisión. Está basado en la técnica del bagging. El número de árboles definidos, B, no hará que se llegue a un sobre ajuste, aunque un número elevado de árboles demasiado crecidos sí puede provocar que los datos tengan varianza innecesaria. El control de la profundidad de los árboles mejora ligeramente el error cometido: a más profundos sean, más error.

La pérdida de correlación entre los diferentes árboles que forman el bosque se consigue mediante la selección de m variables de forma aleatoria de las p posibles que tienen los datos (normalmente $m = \sqrt{p}$ en clasificación y $m = p/3$ en regresión). Cada uno de los árboles escoge la mejor combinación de las m variables para la muestra “Bootstrap-eada” de datos con la que se está entrenando. El decremento del valor de m responderá a una menor varianza, sin embargo, es posible que aumente el sesgo debido a que se incrementan las restricciones del modelo sobre los datos (cada árbol tiene menos variables para modelar los datos).

En regresión, la salida del bosque aleatorio consistirá en (idéntico a bagging)

$$\hat{f}_{rf}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b\,(x)$$

La media de las B variables aleatorias independientes e idénticamente distribuidas, $\frac{1}{B}\sum^{B} X_i$, con varianza σ^2 cada una, es $\frac{1}{B}\sigma^2$.

Si las variables son simplemente idénticas en distribución (como ocurre en los bosques aleatorios) entonces la varianza sería:

$$\begin{aligned}\mathrm{Var}\left[\frac{1}{B}\sum^{B} X_i\right] &= \mathrm{E}\left[\left(\frac{1}{B}\sum^{B} X_i - \mathrm{E}\left[\frac{1}{B}\sum^{B} X_i\right]\right)^2\right] = \mathrm{E}\left[\frac{1}{B^2}\cdot\left(\sum^{B} X_i - \sum^{B} \mu_i\right)^2\right] = \frac{1}{B^2}\mathrm{E}\left[\sum^{B}(X_i-\mu_i)\sum^{B}(X_j-\mu_j)\right]\\ &= \frac{1}{B^2}\sum_{i=1}^{B}\sum_{j=1}^{B}\mathrm{E}\left[(X_i-\mu_i)(X_j-\mu_j)\right] = \frac{1}{B^2}\sum_{i=1}^{B}\sum_{j=1}^{B}\mathrm{Cov}\left(X_i, X_j\right) = \frac{1}{B^2}\left(B\sigma^2 + B(B-1)\rho\sigma^2\right)\\ &= \boldsymbol{\rho\sigma^2 + \frac{\sigma^2}{B}(1-\rho)}\end{aligned}$$

Por lo que, a más se incremente el número de árboles, $B \to \infty$, más dependerá la salida del bosque aleatorio de cuanto de no correlacionados estén los árboles entre ellos, pues $\frac{\sigma^2}{B}(1-\rho) \to 0$, y solo queda el primer término.

- **Error out-of-bag**: cada árbol bootstrap usa, de media, 2/3 de los datos, de forma que el 1/3 restante se usa para calcular el error, que se realiza de forma simultánea al entrenamiento del modelo.

Conocer la importancia de las variables en los bosques aleatorios es más complejo que en los árboles debido al bagging. Para ellos existen dos medidas principalmente:

- **Impureza de Gini**. Cada vez que una variable se usa para dividir un nodo y se mejora la impureza de Gini, se acumula esa mejora sobre todos los árboles.
- **Permutación**. Tras entrenar, se permutan al azar los valores de una variable en los datos OOB y se mide cuánto empeora la precisión.

Tema 2. Intensificación (boosting)

1. Introducción

La intensificación sirve para construir un modelo fuerte combinando muchos modelos débiles (como los árboles de decisión pequeños) de forma secuencial. Estos modelos débiles producen una hipótesis ligeramente mejor que un predictor naive (acierto del 50%). Es decir, la intensificación se basa en la asunción de la hipótesis débil: el error que cometerá el modelo débil será, como mucho, $\frac{1}{2} - \gamma$, para $\gamma > 0$.

2. AdaBoost

Es un algoritmo de intensificación basado en llamadas iterativas al modelo base.

AdaBoost se caracteriza por mantener una distribución, D, sobre los datos de entrenamiento que se actualiza en cada iteración t, D_t. Cada instancia de entrenamiento, x_i, junto con su etiqueta, y_i, tiene un peso dado por dicha distribución $D_t(x_i)$. En $t = 1$, la distribución es uniforme, pues se inicializa D tal que: $D_1(x_i) = 1/m, \forall x_i$, con m el número de instancias. Esta distribución se interpreta como un peso y mide cómo de incorrecta ha sido clasificada la instancia a lo largo del entrenamiento, de forma que el modelo base centre más su atención en ellos.

En cada iteración se desea encontrar un clasificador $h_t : X \rightarrow \{-1, +1\}$ apropiado para la distribución que se tenga en la iteración D_t.

A partir del clasificador obtenido se calcula ϵ_t, que es la probabilidad de, escogiendo un elemento x_i aleatorio en función de la distribución D_t, $h_t(x_i)$ no lo clasifique bien. Es decir:

$$\epsilon_t \doteq \Pr_{i \sim D_t}[h_t(x_i) \neq y_i] = \sum_{i: h_t(x_i) \neq y_i} D_t\,(x_i)$$

El error ϵ_t es la suma de los pesos $D_t(x_i)$ tales que x_i no son clasificados correctamente por el clasificador. El modelo tenderá a escoger un clasificador h_t con un error ϵ_t bajo.

A menor sea ϵ_t más importancia debe tener el clasificador h_t a la hora de generar la salida del modelo, por lo que se calcula α_t (inversamente proporcional a ϵ_t):

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

Una vez se tiene este coeficiente, se debe actualizar la distribución D_t para que el algoritmo se centre en aquellas instancias más complejas de clasificar. La forma de actualización de la distribución viene dada por:

$$D_{t+1}(x_i) = \frac{D_t(x_i)}{Z_t} \times \begin{cases} e^{-\alpha_t} \text{ if } h_t(x_i) = y_i \\ e^{\alpha_t} \text{ if } h_t(x_i) \neq y_i \end{cases}$$

El error que comete h_t con respecto a la distribución D_{t+1} es de exactamente ½.

Finalmente, tras T iteraciones, AdaBoost combina la salida de todos los clasificadores h_t para generar una única salida.

$$H(x_i) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t \, h_t(x_i) \right)$$

Esta concatenación de iteraciones creando diferentes clasificadores permite representar espacios no lineales con modelos base que no son capaces de ello. Un exceso en el número de iteraciones (modelos generados) puede provocar un sobre ajuste del modelo final.

2.1. Caso concreto para los árboles

Los árboles de regresión/clasificación dividen el espacio de las instancias en M regiones disjuntas (su intersección es $\emptyset$). El árbol, cuando reciba una instancia x_i que pertenezca a la región R_j, devolverá una constante asociada a dicha región, γ_j.

$$x_i \in R_j \Rightarrow f(x) = \gamma_j$$

El valor γ_j dependerá de la finalidad del árbol: si es regresión generalmente es la media de las instancias que pertenecen a R_j, $\gamma_j = \overline{y_i}$; mientras que si es clasificación devolverá la clase a la que pertenecen la mayoría de las instancias que pertenecen a dicha región. En el caso de la clasificación, se define el árbol como:

$$T_\Theta(x_i) = \sum_{j=1}^{J} \gamma_j \, I\left(x_i \in R_j\right)$$

Con Θ los parámetros del árbol y la función I una función indicadora (si se cumple la condición $I(\ldots) = 1$, sino 0). Θ serán los parámetros que hagan mínima la función de pérdida entre las diferentes etiquetas asociadas a cada instancia y la clasificación que hace el árbol.

La complejidad reside en estimar las regiones. Generalmente usando un algoritmo voraz (**greedy**) de partición de arriba abajo (de más general a más específicas las regiones).

En el caso de los árboles intensificados (**Boosted Tree Model**), se expresa la salida obtenida por:

$$f_M(x_i) = \sum_{m=1}^{M} T_{\Theta_m}(x_i)$$

Con M el número total de iteraciones. En este caso, los argumentos del árbol en el momento m son los que hacen mínimo la función de pérdida entre las etiquetas reales y la predicha por la combinación entre los modelos anteriores y el actual.

$$\hat{\Theta}_m = \arg\min_{\Theta_m} \sum_{i=1}^{N} L\left(y_i, f_{m-1}(x_i) + T_{\Theta_m}(x_i)\right)$$

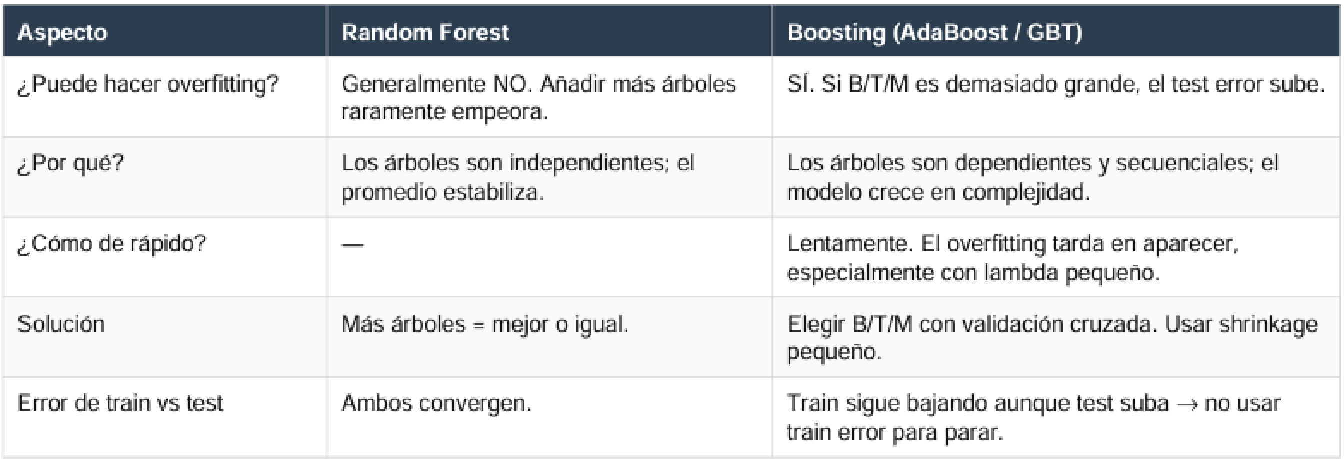
Si en lugar de hacerse con todas las etiquetas se hacen solo con aquellas instancias que pertenecen a una clase, entonces se está obteniendo el valor a devolver por el modelo.

$$\hat{\gamma}_{jm} = \arg\min_{\gamma_{jm}} \sum_{x_i \in R_{jm}} L\left(y_i, f_{m-1}(x_i) + \gamma_{jm}\right)$$

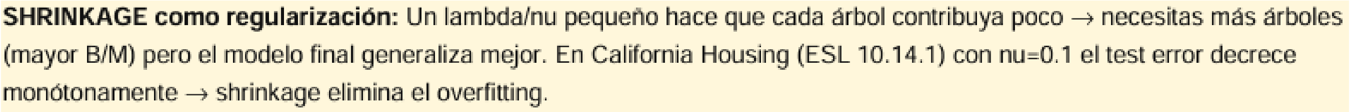
Las funciones de pérdida utilizadas dependerán del objetivo para el que se use el árbol:

- Regresión: error cuadrático, error absoluto y Huber (estos dos para regresión robusta, requiriendo la intensificación del gradiente).
- Clasificación binaria: exponencial y deviance (logística). Esta última requiere la intensificación del gradiente.
- Clasificación multinomial: deviance multinomial.

Diferencias con respecto a Random Forest:



Aspecto	Random Forest	Boosting (AdaBoost / GBT)
¿Puede hacer overfitting?	Generalmente NO. Añadir más árboles raramente empeora.	SÍ. Si B/T/M es demasiado grande, el test error sube.
¿Por qué?	Los árboles son independientes; el promedio estabiliza.	Los árboles son dependientes y secuenciales; el modelo crece en complejidad.
¿Cómo de rápido?	—	Lentamente. El overfitting tarda en aparecer, especialmente con lambda pequeño.
Solución	Más árboles = mejor o igual.	Elegir B/T/M con validación cruzada. Usar shrinkage pequeño.
Error de train vs test	Ambos convergen.	Train sigue bajando aunque test suba → no usar train error para parar.



SHRINKAGE como regularización: Un lambda/nu pequeño hace que cada árbol contribuya poco → necesitas más árboles (mayor B/M) pero el modelo final generaliza mejor. En California Housing (ESL 10.14.1) con nu=0.1 el test error decrece monótonamente → shrinkage elimina el overfitting.

3. Intensificación del gradiente

AdaBoost no puede trabajar con cualquier función de pérdida, mientras que la intensificación del gradiente sí que puede hacerlo siempre y cuando sea diferenciable. Al usar funciones de ese tipo, es necesario usar la optimización numérica para encontrar el mínimo de la función de pérdida dada una función f.

La búsqueda de los parámetros de f que hacen mínima a la función de pérdida L se expresa tal que:

$$\hat{\mathbf{f}} = \arg\min_{\mathbf{f}} L(\mathbf{f})$$

Con $\mathbf{f} = \{f(x_1), \dots, f(x_N)\}^T$. Esta ecuación se resuelve tal que:

$$\mathbf{f}_M = \sum_{m=0}^{M} \mathbf{h}_m$$

En el algoritmo del descendente más inclinado (**steepest descent**), el valor $\mathbf{h}_m$ se calcula tal que:

$$\mathbf{h}_m = -\rho_m \mathbf{g}_m$$

Siendo ρ_m un escalar y $\mathbf{g}_m$ el gradiente de $L(\mathbf{f}_{m-1})$. Cada componente g_{im} del gradiente se define como

$$g_{im} = \left[\frac{\partial L\big(y_i, f_{m-1}(x_i)\big)}{\partial f_{m-1}(x_i)}\right]$$

Por lo que el término $\mathbf{f}_m$ se actualiza tal que:

$$\mathbf{f}_m = \mathbf{f}_{m-1} - \rho_m \mathbf{g}_m$$

Haciendo que la función de pérdida decrezca rápidamente. Este método es el gradiente descendente. Solo puede calcularse sobre el conjunto de entrenamiento, por lo que se hará durante este.

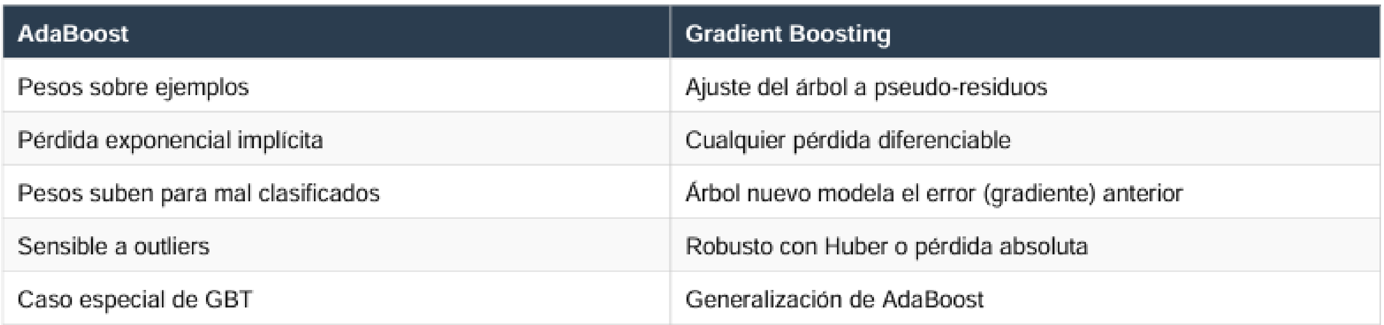
Según la función de perdida establecida, se obtendrán un tipo de residuos u otros y, con ellos, el tipo de intensificación. Con la función de perdida cuadrática, los pseudo-residuos (g_{im}) coinciden con los ordinarios ($y_i - f(x_i)$).

Hiperparámetros:

- **Número de árboles (M)**. Ajustan mejor el entrenamiento, pero puede provocar sobreajuste.
- **Tasa de aprendizaje**. Multiplica la contribución de cada árbol. Valores pequeños regularizan.

- **Tamaño del árbol**. Cantidad de niveles. Controla el orden de las interacciones.
- **Submuestreo**. Uso de una fracción aleatoria de los datos en cada árbol.

Diferencias con respecto a AdaBoost:



AdaBoost	Gradient Boosting
Pesos sobre ejemplos	Ajuste del árbol a pseudo-residuos
Pérdida exponencial implícita	Cualquier pérdida diferenciable
Pesos suben para mal clasificados	Árbol nuevo modela el error (gradiente) anterior
Sensible a outliers	Robusto con Huber o pérdida absoluta
Caso especial de GBT	Generalización de AdaBoost

3.1. Importancia relativa de las variables predictoras

La medida propuesta es, para un árbol T, la importancia relativa.

$$\mathcal{J}_l^2(T) = \sum_{t=1}^{J-1} \hat{\imath}_t^2 \, I(l = l_t)$$

Se recorren los $J-1$ nodos internos del árbol y, en cada nodo t, se calcula cuál es la variable de los datos, l_t, que es la que particiona la región asociada a ese nodo de forma que se consiga máxima mejora estimada, $\hat{\imath}_t^2$, en el MSE. Para cada una de las variables l se obtiene que $\mathcal{J}_l^2(T)$ es la mejora general que se consigue en el árbol T. Es necesario iterara sobre todos los árboles para conocer la importancia de la variable en concreto:

$$\mathcal{J}_l^2 = \frac{1}{M} \sum_{m=1}^{M} \mathcal{J}_l^2\,(T_m)$$

En el caso de que se tenga árboles clasificadores multiclases (K clases), se tiene una importancia a partir del total de los árboles por cada una de las clases, $\mathcal{J}_{lk}^2$, que posteriormente se usa para obtener la importancia general, $\mathcal{J}_l^2$.

A mayor $\mathcal{J}_l^2$, mayor importancia.

3.2. Partial Dependence Plots

Los gráficos de dependencia parcial muestran el efecto marginal de una variable sobre la predicción. Dadas el conjunto de características $V = \{v_1, v_2, \dots, v_l\}$, tomemos $S \subset V$, y su complementario en V, S'. Se puede estimar la dependencia parcial de una función f sobre el conjunto S tal que:

$$\bar{f}_S(S) = \frac{1}{N} \sum_{i=1}^{N} f\,(S, x_{iS'})$$

Donde $x_{iS'}$ son los valores de las variables de S' que ocurren en el conjunto de entrenamiento. Se requiere que se evalúe para conjunto de valores de S. Es costoso pero eficiente en árboles.

Diferencia CLAVE entre PDP y condicionado

PDP f_bar_S(Xs) [usar esto]	Condicionado f_tilde_S = E[f|Xs] [NO usar]
Efecto de Xs TRAS promediar (marginalizar) efectos de Xc	Mejor aprox. de f ignorando Xc — equivale a E[f(X)|Xs]
Son iguales SOLO si Xs perpendicular Xc	Puede mostrar efectos fuertes aunque f no dependa de Xs
Interpretación: prima extra que se paga por Xs	Mezcla el efecto de Xs con la correlación con Xc

- Útil para detectar **interacciones** entre variables (PDP 2D, ej. HouseAge × AveOccup en ESL 10.14.1).
- Para K-clase: PDP sobre fk(X) da la **log-odds** de la clase k como función de las variables.
- Limitación práctica: solo subsets pequeños (l ≈ 1, 2, 3 variables).

EXAMEN: PDP es una herramienta de interpretación de **caja negra** — funciona con cualquier método de aprendizaje, no solo boosting. Las curvas PDP en árboles no son suaves (modelos constantes a trozos) pero muestran tendencias reales.

Tema 3. Combinaciones de modelos

1. Marco general

La combinación de modelos (clasificadores) se reduce a utilización de diferentes modelos, cada uno generando una salida, y combinando esas salidas para la obtención de la mejor posible. El objetivo es encontrar el algoritmo óptimo para realizar la combinación de la salida.

En el caso de la clasificación con N clases, el algoritmo combinatorio recibirá un vector por cada clasificador, cada uno con las puntuaciones de la instancia de entrada para cada una de las clases. Este patrón de entrada se representa como x_i en cada uno de los M clasificadores. A la instancia, representada por todos los x_i vectores se le asignará la clase n que produzca la máxima probabilidad a posteriori. Es decir, la clase con mayor probabilidad dados todos los vectores.

Los clasificadores pueden ser el mismo modelo, pero que usen diferentes conjuntos de características o configuración; o diferentes modelos, pero con el mismo conjunto de características. Es importante que los clasificadores no cometan los mismos errores en las mismas instancias.

2. Estrategias para la combinación de clasificadores

2.1. Combinación por nivel

Se puede dar la combinación a nivel de datos, a nivel de características o a nivel de decisión.

A nivel de datos puede darse la unión de datos provenientes de diferentes fuentes, provocando en algunas ocasiones un número elevado de características en caso de que se realice una concatenación de características. Para ello, los vectores pueden reducirse con técnicas como PCA.

A nivel de decisión existen tres enfoques:

- **Abstracto**. Cada clasificador devuelve una única etiqueta al algoritmo combinador.
- **Ranking**. Cada clasificador devuelve una lista de etiquetas de más a menos probables.
- **Puntuación**. Cada clasificador devuelve las n mejores etiquetas junto con una puntuación de confianza. Se suele usar la teoría Bayesiana o la teoría de la evidencia (D-S).

La teoría de la evidencia es una generalización de la teoría Bayesiana. La diferencia entre ambos es la incorporación de la incertidumbre. Se basa en tres conceptos principales: asignación de creencia básica, función de creencia y plausibilidad.

2.2. Combinación de clasificadores débiles y fuertes

La combinación dura o fuerte usa la salida de los clasificadores después de usar un umbral duro. Por ejemplo, la votación por mayoría, unánime o mayor número de votos. En este caso, si existe pruebas de que las precisiones de los clasificadores son diferentes, se puede ponderar la decisión de los clasificadores.

La combinación suave o débil usa estimaciones de las probabilidades a posteriori. Por ejemplo, las reglas de la suma, el mínimo, máximo, mediana, etc.

2.3. Combinación adaptativa y no adaptativa

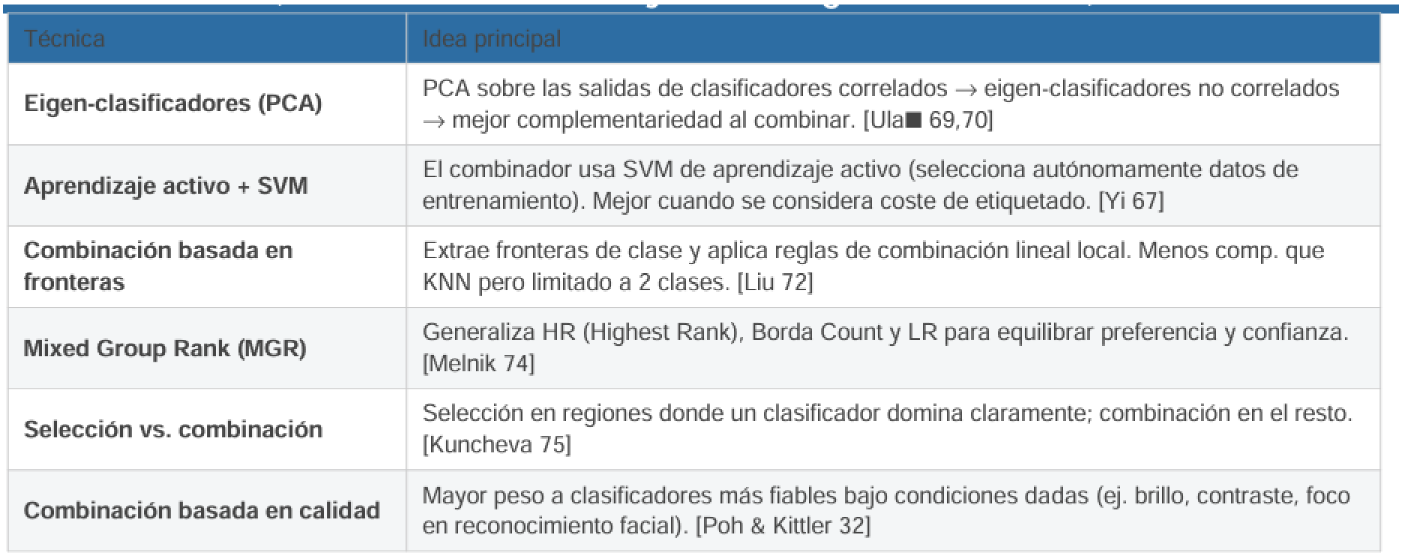
Las combinaciones adaptativas se basan principalmente en algoritmos evolutivos, de inteligencia artificial o de lógica difusa. Uno de los métodos es el stacking o la generalización apilada, que consiste en una arquitectura de dos niveles, donde varios modelos producen predicciones y un metamodelo que se entrena sobre esas predicciones para emitir la salida final. Este modelo aprende patrones de error y correlaciones entre los modelos bases. Si el metamodelo es la media directa, el stacking se comporta como bagging; si por el contrario es lineal, entonces no puede captar interacciones complejas.

En las combinaciones no adaptativas entran los algoritmos de alta confianza, los de voto, conteo Borda o adición de vectores.

2.4. Clasificación basada en el número de clasificadores

El artículo añade una última distinción según se combinen pocos clasificadores (< 10) o muchos; el caso de muchos coincide ya con el territorio de bagging y boosting (temas 1 y 2).

2.5. Otras técnicas.



Técnica	Idea principal
Eigen-clasificadores (PCA)	PCA sobre las salidas de clasificadores correlados → eigen-clasificadores no correlados → mejor complementariedad al combinar. [Ula■ 69,70]
Aprendizaje activo + SVM	El combinador usa SVM de aprendizaje activo (selecciona autónomamente datos de entrenamiento). Mejor cuando se considera coste de etiquetado. [Yi 67]
Combinación basada en fronteras	Extrae fronteras de clase y aplica reglas de combinación lineal local. Menos comp. que KNN pero limitado a 2 clases. [Liu 72]
Mixed Group Rank (MGR)	Generaliza HR (Highest Rank), Borda Count y LR para equilibrar preferencia y confianza. [Melnik 74]
Selección vs. combinación	Selección en regiones donde un clasificador domina claramente; combinación en el resto. [Kuncheva 75]
Combinación basada en calidad	Mayor peso a clasificadores más fiables bajo condiciones dadas (ej. brillo, contraste, foco en reconocimiento facial). [Poh & Kittler 32]

Tema 4. Aprendizaje no supervisado

1. Introducción

Consiste en la inferencia de las propiedades de las instancias sin necesidad de ningún indicador. En estos problemas se suele elegir el algoritmo según el número de características se tenga. Generalmente los algoritmos buenos para un número bajo de características no lo son cuando se tienen muchas. Los algoritmos destinados a un número elevado de características suelen tener de forma implícita la búsqueda de una representación de menor dimensionalidad.

La evaluación de estos métodos es una cuestión de opinión y no puede ser verificada directamente debido a la inexistencia de etiquetas.

2. Análisis de grupos

También se puede denominar segmentación de datos (data segmentation) si los datos no son tan fáciles de separar.

La agrupación de instancias se puede realizar según las propiedades o por su relación con otros objetos. Una correcta diferenciación de instancias puede evaluarse si los grupos representan objetos con propiedades lo suficientemente diferentes. A esto se le denomina similitud. Esta similitud suele medirse en base a la distancia medida entre dos objetos.

2.1. Medidas de distancia

Cuando los datos son presentados directamente según la distancia entre ellos o mediante la afinidad o similitud, se dice que están presentados en **matrices de proximidad**. Generalmente las posiciones m_{ij} de la matriz son valores de “no similitud” (de forma que la diagonal es 0, pues un objeto es idéntico a sí mismo). Si se presentase el caso contrario sería conveniente transformar la matriz. De igual forma, también se espera que esta matriz sea simétrica, por lo que si no lo es habría que reemplazarla por $\left(\mathbf{D}+\mathbf{D}^2\right)/2$. La información que contienen estas matrices raramente son distancias estrictas, pues la desigualdad triangula no se da siempre.

Otra forma de medir la similitud entre instancias es mediante el uso de sus características. Se define la diferencia entre dos instancias, x_i y x_j, como la suma de la diferencia entre todas sus características, k:

$$D(x_i, x_j) = \sum_{k=1}^{p} d_k\left(x_i^k, x_j^k\right)$$

La más común es la distancia cuadrada $\left(x_i^k - x_j^k\right)^2$, que mide la diferencia (a más lejanas más diferentes). Según el tipo de característica, se tendrá mediadas diferentes:

- **Características cuantitativas**. Junto con la distancia cuadrada también se usa la distancia absoluta o identidad. También se puede usar la correlación, que mide la similitud.

  $$\rho(x_i, x_j) = \frac{\sum_k\left(x_i^k - \bar{x}_i\right)\left(x_j^k - \bar{x}_j\right)}{\sqrt{\sum_k\left(x_i^k - \bar{x}_i\right)^2 \sum_k\left(x_j^k - \bar{x}_j\right)^2}}$$

  Con $\bar{x}_i$ la media de las características cuantitativas de la instancia x_i. Si se estandariza las características cuantitativas, entonces la correlación pasa a actuar como la distancia cuadrada.
- **Características ordinales**. Se pueden tratar como características cuantitativas si se transforma la categoría ordinal (cada una tiene un valor del 1 al M según el ranking) por el valor $\frac{1-1/2}{M}$.

- **Características categóricas o nominales**. No existe una medida como tal entre los valores de este tipo de características, por lo que se suele interpretar que si no son la misma clase entonces la distancia es 1 y si sí lo son es 0.

Normalmente la importancia de cada característica es diferente a la hora de calcular la diferencia entre las instancias. Para ello se introduce un peso w_k que pondera $d_k\left(x_i^k, x_j^k\right)$. Las variables más relevantes en la agrupación de los datos deben tener un w_k superior. Si se desea que todas las variables tengan la misma influencia en el cálculo de la distancia se deberá indicar $w_k \sim \frac{1}{\bar{d}_k}$, donde $\bar{d}_k$ es:

$$\bar{d}_k = \frac{1}{N^2}\sum_{i=1}^{N}\sum_{j=1}^{N} d_k\left(x_i^k, x_j^k\right)$$

Que es la diferencia media de las características en cuestión. Si en este caso d_K es el error cuadrático, $\bar{d}_k = 2 \cdot \mathrm{var}_k$ (el doble la estimación de la varianza de la característica k). Cuando existen instancias con características faltantes se puede optar por excluir a esas características del cálculo de la distancia o realizar una interpolación mediante la media/mediana o mediante la creación de una nueva clase “desconocido” si se puede considerar que los objetos que les falte esa medición son parecidos.

2.2. Algoritmos de agrupamiento

Existen tres tipos de algoritmo de agrupamiento:

- **Algoritmos combinatorios**. Trabaja directamente en los datos observados sin referencia directa en el modelo probabilístico inferior.
- **Modelos de mezclas**. Supone que los datos son independientes e idénticamente distribuidos. El modelo consiste en diferentes distribuciones (normalmente gaussianas), cada una con sus parámetros, capaces de modelas a los datos.
- **Búsqueda de modo (mode seeking)**. Intenta estimar directamente las modas de la función de densidad. Por ejemplo, PRIM o mean-shift.

En este tipo de modelos, la cross-validacion no es útil. Se utiliza el estadístico GAP, que busca el codo en la curva de W_k (es decir, el mejor número de grupos según la evolución del error) de forma automática.

2.3. Algoritmos combinatorios

Dados K clústeres (con K como máximo el número de instancias menos 1) se pretende asignar cada clúster k a cada instancia x_i, $C(x_i) = k$. Para conseguir la asignación óptima en base a la distancia utilizada es necesario minimizar

$$W(C) = \frac{1}{2}\sum_{k=1}^{K}\sum_{C(x_i)=k}\sum_{C(x_j)=k} d\left(x_i, x_j\right)$$

Es decir, la distancia entre elementos del mismo clúster. Esta distancia también se puede expresar como

$$W(C) = T + B(C)$$

Donde T es la suma de las distancias entre cada para de instancias independientemente del clúster asignado (valor constante en el conjunto de datos), y $B(C)$ es el caso contrario a $W(C)$, la suma de las instancias tal que $C(x_i) = k$ y $C\left(x_j\right) \neq k$. La minimización de $W(C)$ es la maximización de $B(C)$.

Este algoritmo puede tener un coste computacional elevado, por lo que lo adecuado es hacer una partición inicial que consideremos que es cercana a la óptima, aunque puede ser un óptimo local.

2.4. K-medias

Es ideal para situaciones donde todas las características son cuantitativas. Generalmente se usa la distancia euclídea.

Para cada clúster se define un centroide, $\bar{x}_k$, como la media cada una de las características que forman parte de ese clúster, $\left(\bar{x}_k^1, \bar{x}_k^2, \ldots, \bar{x}_k^p\right)$. En cada iteración se recalcula la distancia de cada instancia x_i con cada uno de los $\bar{x}_k$ y se reajusta los clústeres cambiando la asignación de las instancias al clúster con el centroide más cercano. Tras realizar la reasignación se vuelve a calcular los centroides. El problema de minimización es:

$$W(C) = \frac{1}{2}\sum_{k=1}^{K} \sum_{C(x_i)=k} \sum_{C(x_j)=k} \left\| x_i - x_j \right\|^2$$

Esta minimización converge a un mínimo, aunque no se asegura que sea global.

Existen diferentes derivaciones de este algoritmo:

- **Cuantización de vectores**. En este caso se usa el algoritmo, denominado algoritmo de Lloyd, para comprimir imágenes. Se usan los centroides para determinar qué regiones son parecidas, de forma que se puedan usar para almacenar la información de la imagen (con cierta pérdida de detalle) y poder reconstruirla más adelante.
- **K-medoids**. Se trata del mismo algoritmo que K-medias, solo que en este caso los clústeres vienen dados por instancias reales de los datos, no por medias de los datos que conforman el centroide. En este caso se parten de K medoids y se va modificando cada uno conforme se vaya reduciendo la distancia con el resto de las instancias dentro del clúster hasta que se alcance un mínimo. En las sucesivas iteraciones se irán modificando la conformación de los clústeres.

Dependiendo del caso, el valor K puede también ser parte del problema de optimización, como ocurre en la segmentación. En la búsqueda del mejor valor de K se tenderá a encontrar mejores resultados a mayor sea el valor. Intuitivamente debe existir un K^* real que agrupe los objetos de forma óptima, por lo que para cualquier valor de $K < K^*$ se tendrá un subconjunto de instancias bien clasificadas. Al llegar K al valor real K^* se verá como el error deja de decrecer tan notablemente o provoca una “curva”. Es decir:

$$\{W_K - W_{k+1} : K < K^*\} \gg \{W_k - W_{k+1} : K > K^*\}$$

2.5. Mezclas gaussianas

Se trata de una derivación del modelo k-medias usando K distribuciones gaussianas basándose en el algoritmo EM (expectación-maximización). Cada instancia tiene una probabilidad de pertenecer a una de las densidades, por lo que se trata de un algoritmo de asignaciones blandas. En cada iteración se actualizan las probabilidades y se vuelve a calcular las distribuciones, de forma que se alcanza un mínimo local.

2.6. Agrupamiento jerárquico

Consiste en la agrupación (algoritmos aglomerativos o bottom-up) o en la división (algoritmos divisivos o top-down) en los que las instancias están en un determinado clúster según el nivel en el que se esté.

En los algoritmos aglomerativos la decisión para la creación de cada clúster por nivel es según la medida de similitud o diferencia existente entre las instancias. El árbol que se forma al dibujar los diferentes niveles en los que se van agrupando las instancias se denomina dendograma.

Los grupos que se unan en niveles altos serán candidatos a ser grupos naturales. Es posible que un grupo sea subconjunto de otro más grande. También se pueden generar grupos que no existan en la realidad.

Las métricas de diferencia entre dos grupos, H y G, pueden ser:

- **Single linkage (vecino más cercano)**. La diferencia entre los dos grupos es la diferencia mínima existente entre una instancia de H y una de G.

  $$d_{SL}(G,H) = \min_{x_i \in G; x_j \in H} d_{x_i,x_j}$$

  Tendencia a combinar con umbrales bajos. Los clústeres formados con esta métrica tienen a violar la compacidad. Depende solo de la ordenación de d_{x_i,x_j}.
- **Complete linkage (vecino más lejano)**. La diferencia entre los dos grupos es la diferencia entre las instancias más lejanas de H y G.

  $$d_{CL}(G,H) = \max_{x_i \in G; x_j \in H} d_{x_i,x_j}$$

  Efecto contrario al vecino más cercano: los clústeres violan la cercanía, se forman con muy poco diámetro. Depende solo de la ordenación de d_{x_i,x_j}.
- **Group average**. La diferencia entre los dos grupos es la diferencia media entre todo par de instancias de cada uno de los dos grupos.

  $$d_{GA}(G,H) = \frac{1}{N_G N_H} \sum_{x_i \in G} \sum_{x_j \in H} d_{x_i,x_j}$$

  Depende de la escala numérica de los datos.

En los algoritmos divisorios se parte de un grupo con todas las instancias y se va separando en otro grupo las instancias más diferentes hasta que la diferencia con el nuevo grupo sea mayor que la diferencia con el actual. Se repite este procedimiento en los dos grupos resultantes hasta que se quiera o se llegue a un grupo por instancia.

3. Métodos para la evaluación de agrupaciones

Las propiedades deseables de una agrupación son tres:

- **Invarianza a la escala**. $f(d) = f(\alpha \cdot d)$ para cualquier función de distancia d.
- **Riqueza**. La función agrupadora debe ser lo suficientemente flexibles como para producir cualquier clúster arbitrario dado el conjunto de datos de entrada.
- **Consistencia**. Los clústeres no cambian sea cual sea la función de distancia que se utilice.

No existe función de agrupación que satisfaga las tres condiciones. Se pueden romper estas tres propiedades de forma:

- Parando el algoritmo cuando se tengan k clústeres. Se pierde la riqueza.
- Parando el algoritmo cuando se tienen que la pareja más cercana es mayor que cierto umbral. Se pierde la invarianza de la escala.
- Parando el algoritmo cuando el par de clústeres más cercano están más lejos que una fracción ϵ de la distancia máxima entre pares. Se pierde la consistencia.

Aspectos para tener en cuenta cuando se realiza la evaluación de los grupos:

- Tendencia del agrupamiento.
- Número correcto de clústeres.
- Asegurar la calidad del clúster sin información externa.
- Comparar los resultados con información externa.
- Comparar dos conjuntos de grupos para determinar el mejor.

Las tres primeras se realizan mediante validación interna o no supervisada. Las siguiente mediante validación externa o supervisada. La últimas mediante ambas validaciones.

3.1. Validación interna

Se basan en propiedades internas de los clústeres: la compactibilidad (que tan similares y juntos están los objetos de un mismo clúster) y la separación (que tan alejados están los clústeres entre sí).

$$cohesion(C_i) = \sum_{x \in C_i, y \in C_i} proximity(x, y)$$

$$separation(C_i, C_j) = \sum_{x \in C_i, y \in C_j} proximity(x, y)$$

La cohesión es equivalente a la métrica SSE (suma de los errores cuadráticos). A menor valor mejor. Por otro lado, la separación entre clústeres es medida por la métrica SSB, que mide la separación entre clústeres. A mayor valor mejor.

Diferentes métricas:

- **Coeficiente de Calisnki-Harabasz**. Mide la dispersión interna entre clústeres y la dispersión externa.

$$CH = \frac{\frac{SSB_M}{M-1}}{\frac{SSE_M}{M}}$$

- **Índice de Dunn**. Ratio entre la menor distancia entre datos de diferentes clústeres y la mayor distancia entre clústeres.

$$D = \min_{1<i<k} \left\{ \min_{1<j<k, i \neq j} \left\{ \frac{\delta\left(C_i, C_j\right)}{\max_{1<l<k} \left\{ \Delta\left(C_l\right) \right\}} \right\} \right\}$$

- **Puntuación de Xie-Beni**. Diseñado para agrupamiento difuso pero aplicable para agrupamiento duro. Ratio entre el nivel de compactibilidad de los datos en un mismo cluster y el nivel se separación de los datos de diferentes clústeres.

$$XB = \frac{\sum_{i=1}^{N} \sum_{k=1}^{M} u_{ik}^2 |||x_i - C_k|||^2}{N_{t \neq s} \min\{|||C_t - C_s|||^2\}}$$

- **Índice Ball-Hall**. Dispersión basada en las distancias cuadráticas de los puntos del cluster con respecto a su centroide.

$$BH = \frac{SSE_M}{M}$$

- **Índice de Hartigan.** Logaritmo de la división entre SSB y SSE.

$$H = log\left(\frac{SSB_M}{SSE_M}\right)$$

- **Coeficiente de Xu.**

$$Xu = D \cdot \log_2\left(\sqrt{\left(\frac{SSE_M}{D \cdot N^2}\right)}\right) + \log M$$

- **Silueta**. Coeficiente más usado. Se calcula con respecto a un solo punto. Es necesario tres métricas:
  - La distancia media desde ese punto a todos los demás en el mismo clúster.

$$a(i) = \frac{1}{|C_a|} \sum_{j \in C_a, i \neq j} d\,(i, j)$$

  - La distancia media mínima desde ese punto al resto de clústeres.

$$b(i) = \min_{C_b \neq C_a} \frac{1}{|C_b|} \sum_{j \in C_b} d\,(i, j)$$

  - A partir de estas dos métricas se calcula la silueta

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Los valores cercanos 1 indican agrupación densa y fuertemente separados. La silueta de una agrupación se calcula como la media de la silueta de todas sus instancias.

- **Coeficiente de correlación cofenética (CPCC)**. Usado para la evaluación de algoritmos jerárquicos. Es la proximidad a la que el algoritmo pone dos instancias en el mismo clúster por primera vez. Corresponde al nivel en donde las ramas de las respectivas instancias se unen. En el intervalo [-1, 1]. A mayor valor, mejor ha detectado la estructura interna de los datos el algoritmo.
- **Estadístico de Hubert**. Mismo objetivo que CPCC. También en el rango [-1,1]. A mayor valor, mejor agrupamiento.

3.2. Validación externa

Se asocia con la comparación las etiquetas originales con el fin de saber: verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN).

Las diferentes métricas que pueden calcularse son:

- **Precisión.** Cuantas instancias son clasificadas correctamente.

$$p = \frac{TP}{TP + FP}$$

- **Recall**. Cuantas instancias son incluidas en el mismo clúster.

$$r = \frac{TP}{TP + FN}$$

- **Medida-F.** Combinación de la precisión y el recall.

$$F_\alpha = \frac{1 + \alpha}{\frac{1}{p} + \frac{\alpha}{r}}$$

Si $\alpha = 1, F = \frac{\alpha \cdot p \cdot r}{p + r}$.

- **Pureza.** Cuando clústeres contienen elementos de solo una clase.
- **Coeficiente de Jaccard.** Similitud sin los TN.

$$J = \frac{TP}{TP + FP + FN}$$

- **Coeficiente de Rand.**

$$Rand = \frac{TP + TN}{M}$$

- **Coeficiente de Mallows y Folkes.**

$$FM = \sqrt{\frac{TP}{TP + FP} \cdot \frac{TP}{TP + FN}}$$

- **Coeficiente estadístico de Hubert.**
- **Entropía.**

$$H = -\sum_i p_i \left(\sum_j \frac{p_{ij}}{p_i} \log \frac{p_{ij}}{p_i} \right)$$

- **Información mútua.**

$$MI = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{p_i p_j}$$

$pij = nij/n, pi = ni/n$, and $pj = nj/n$.